Chapter 8 Interruption

8.1 Causes of Interruptions

8.2 Control of interruptions

8.3 Identification of the source of the interruption

8.4 Priority of Interruptions

8.5 Other Tips During Interruption Processing

interrupt

- The action of the computer pausing to solve a problem is called an interrupt
  - A computer error prevents the program from running or gives incorrect results.
  - There are more urgent events that need to be prioritized during program execution.

8.1 Reasons for the interruption

- There are many reasons for disruptions：
  - Conditions caused by errors.
  - It is deliberately designed to force the CPU to pause the process in progress and focus on more urgent or priority matters.

8.1 Reasons for the interruption

- Hardware failure
- Program error
- Real-time event monitoring
- Export/Import

8.1.1 Hardware failure

- To ensure that each action is performed correctly, various circuits are often added to computer hardware to detect faults, such as monitoring whether the power supply is normal or performing parity checks when reading memory data.
- When these circuits find abnormal phenomena, such as when the voltage supplied by the power system is low to a certain level (such as when only 85% of the normal voltage remains), because it is too close to the edge of the circuit that cannot operate properly, it will be too low as long as it is slightly unstable, which is already an abnormal state.

8.1.1 Hardware failure

- In order to avoid data loss in registers or memory due to insufficient voltage, it is necessary to pause the programs running on the current CPU and make emergency responses first. Usually at this time, the computer is forced to stop all normal operations and back up the registers that represent the current state of program execution, or even the data in memory, to secondary storage (such as a hard disk), so that the running program state can be "frozen" so that it can continue to run when power is restored without losing half of the original execution due to power failure.

8.1 Reasons for the interruption

- Hardware failure
- Program error
- Real-time event monitoring
- Export/Import

8.1.2 Program error

- When the result of the ALU operation is overflowing：
  - In the form of an interrupt, the CPU is notified in real time that the calculation result is abnormal.
  - The necessary treatment can be carried out by:
  - Force the execution of the abort program.
  - Start an interrupt service routine.

8.1.2 Program error

- When a program attempts to access memory other than a legitimate block during execution：
  - Interrupt the execution of the program.
  - Issue an error message to alert the user.

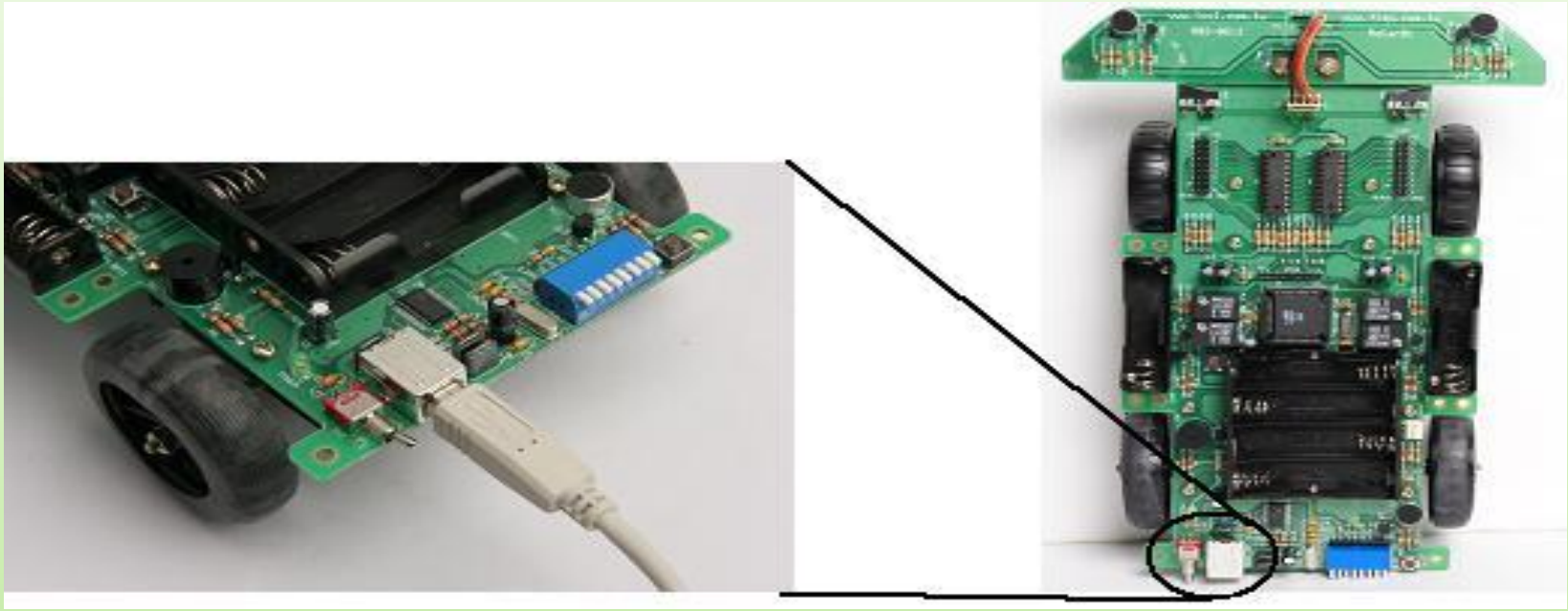
8.1 Reasons for the interruption

- Hardware failure
- Program error
- Real-time event monitoring
- Export/Import

8.1.3 Real-time event monitoring

- In addition to running some commonly used applications, the computer can also be connected to peripherals and monitored and controlled in real time based on the events triggered by these devices.
- For smooth monitoring, signals from external devices are typically received and based on that signal to determine how to respond. These messages are usually used to notify the computer in an outage.

8.1.3 Real-time event monitoring



RoCar Robot Car: Utilizes the computer's interruption mechanism to accept computer control.

8.1 Reasons for the interruption

- Hardware failure
- Program error
- Real-time event monitoring
- Export/Import

8.1.4 Output/Input

- Output and input actions are also one of the common sources of interruption.
- When the computer needs to output or input, the CPU will first issue instructions and make necessary communication to prepare for transmission, and then hand over the transmission action to the transmission channel and peripheral devices for processing.
- After completing the transmission of a certain amount of data, the CPU needs to start the next unit amount of transmission or stop the transmission, so the peripheral device sends an interrupt signal to tell the CPU that it is time to come back to process; The CPU will also stop the running program and go back to processing the output/input actions.

8.2 Interrupt control

- Saving and restoring CPU state
- A simple interrupt system

8.2 Interrupt control

- Interrupting is interrupting a program that the CPU is running and giving control of the executable program to another interrupt routine, which involves a transfer of control.
- It is necessary to design a set of routines that can smoothly hand over control to interrupt the service after the routine is executed, and then smoothly return to the original program where it was interrupted and continue to execute, just as the original program was not interrupted, without any error-free.

8.2.1 Saving and restoring CPU state

- The state of the CPU is actually the entire context of the program when it is executed, including：
  - The contents of all registers.
  - signals on various control lines.
- When the CPU is executing an interrupt service routine, its register properties change accordingly.

8.2.1 Saving and restoring CPU state

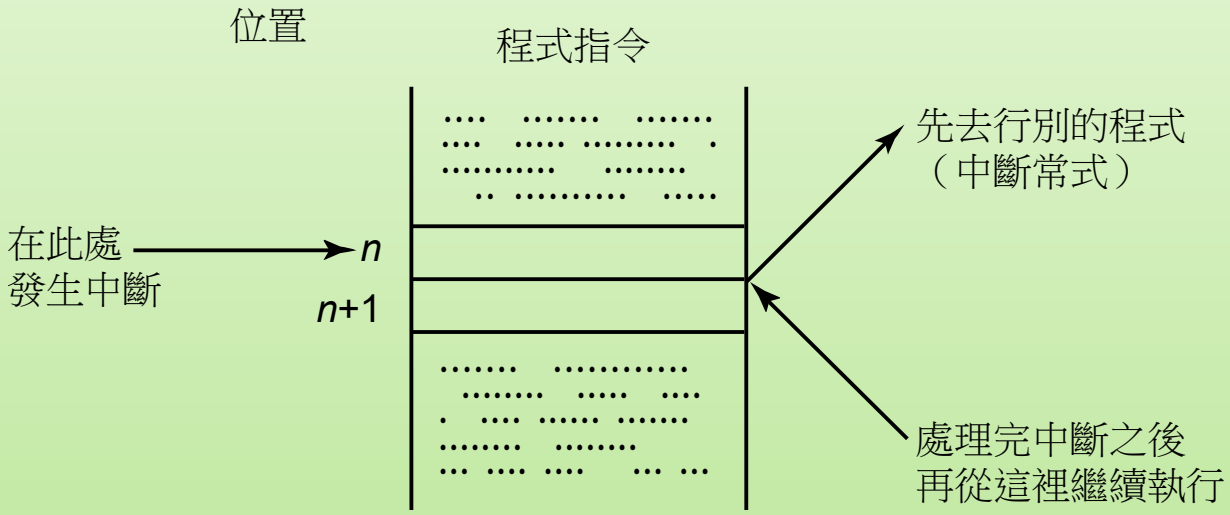
- In order to maintain the original CPU state so that the original program can continue to run as if nothing happened after the interrupt is processed, it is necessary to use specific commands to save the CPU state before starting to enter the interrupt service routine. Before the interrupt service routine ends, these values are stored back in the registers or signal lines.

8.2 Disruption control

- Saving and restoring CPU state
- A simple interrupt system

8.2.2 A simple interrupt system

- **Simple interruption system**：



8.2.2 A simple interrupt system

- The start address of the interrupt service routine must be fixed to a specific address, so that the hardware circuit can directly put it into the PC register to quickly execute the interrupt service routine.
- The interrupt service circuit will definitely recognize two memory addresses：
  - Temporary save the original PC content address.
  - Stores the start address of the interrupt service routine.

	⋮
Interrupt address (address 0) →	Original PC content
（Address 1）→	Interrupt routine address
	⋮

Each circuit from the source of the interrupt has direct access to at least two memory addresses

8.2.2 A simple interrupt system

- Interrupted standard processes：
  1. The CPU first completes the running program instructions.
  2. The hardware does two things：
     a. Save the value of the program count register (PC) to the memory address 0 corresponding to the interrupt.
     b. Load the value recorded in the memory address 1 of the interrupt corresponding to the PC into the PC
  3. The CPU uses the PC as usual to retrieve the next instruction to be executed; What is captured is the first command of the interrupt service routine.

8.3 Identification of the source of the interruption

- Since there are many sources of outages, and the service routines for each outage are different, the first priority in dealing with outages should be to be able to accurately identify the cause of the outage.
- The most straightforward way to identify the cause of an outage：
  - Using hardware lines, each interruption is given a dedicated line.
  - Low flexibility and high hardware design cost make them unsuitable for today's complex computer systems.

8.3 **Identification of the source of the interruption**

- Software Identification Method
- Hardware identification method

8.3.1 Software Identification Method

- software polling
  - The interrupt service routine polls each device one by one in a predetermined order to determine which device is causing an interrupt。
  - Advantage：
    - Simplify hardware circuitry and reduce costs
  - Disadvantage：
    - Slow speed and slow down the performance of the computer

8.3.1 Software Identification Method

- The most streamlined approach is to have the interrupt signal of many devices transmitted over the same line, and a special interrupt service routine that specifically identifies the source of the interrupt determines which device is sending the interrupt signal, and here the interrupt routine corresponding to the interrupt source is called to correctly respond to the interrupt event.

8.3 Identification of the source of the interruption

- Software Identification Method
- Hardware identification method

8.3.2 Hardware identification method

- The commonly used hardware identification system is called vectorized interruption（vectored interrupt）：
  - Provide a code or address for each device that may cause an outage to be identified by the CPU

- Vectorization interrupts the operating process：
  1. A device issues an interrupt request.
  2. The CPU must complete the running program command before sending an acknowledgment message to the device on the interrupted line.

8.3.2 Hardware identification method

3. After receiving an acknowledgment message, the device that made the interrupt request responds with a memory address to the CPU; At this point, the CPU has officially accepted the interrupt request from this device, so it sets the flag of the interrupt request message to Off (0).
4. The CPU first stores the current program counter (PC) content into the received address; Then take a memory address from the next character space of that address, put it into the program counter, and start executing the command therein; It is the first instruction of the device's interrupt service routine.

8.3.2 Hardware identification method

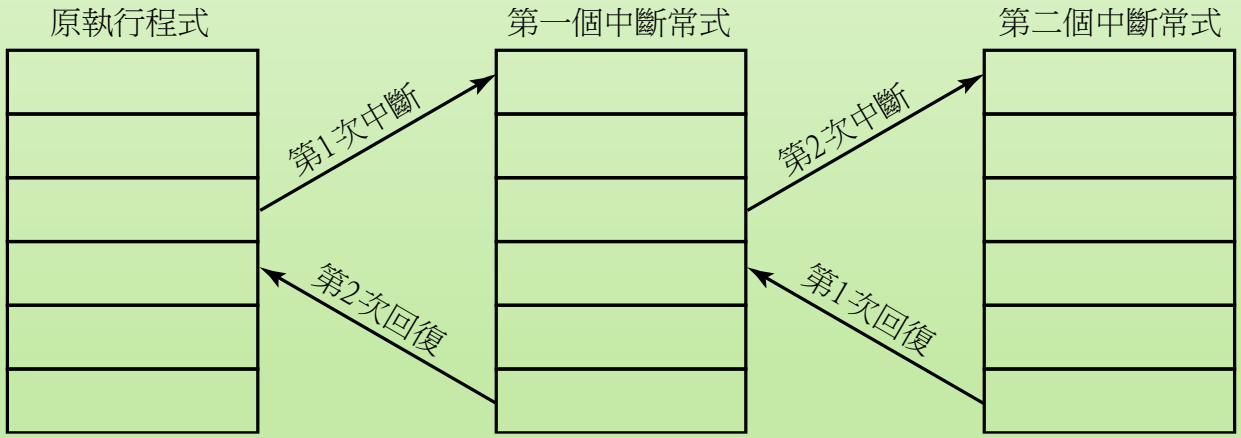
- Hardware identification system：
  - Advantage：
    - Fast execution
  - Disadvantage：
    - More additional circuitry is required.

8.4 Priority of interruptions

- Nested break
- Interruption requests are open and prohibited
- Use software to configure priorities
- Configure priority with hardware
- Interrupt the shield

8.4.1 Nested break

- A nested interrupt, that is, more than two layers of interrupts, occurs when the CPU is still performing the previous interrupt service.



Schematic diagram of nested interruptions

8.4.1 Nested break

- According to the importance of the two sources of interruption, there are two situations：
  - Assuming that the subsequent interrupt is less important than the previous one, we will keep the device with the interrupt flag on and wait for the previous interrupt routine to finish before processing it.

8.4.1 Nested break

- If the interruption is more urgent, then：
  1. The second interrupt interrupts the first interrupt routine and runs the second interrupt routine first.
  2. The first interrupt service routine will be paused, and its next execution command address must be temporarily stored to a specific location.
  3. Complete the second interrupt service routine.
  4. Take out the address of the next command to be executed in the first interrupt service routine and save it back to the program counter to continue to complete the first interrupt service routine.
  5. Take out the address of the next command that should be executed in the original running program, and save it back to the program counter to continue running the program.

8.4 Priority of interruptions

- Nested break
- Interruption requests are open and prohibited
- Use software to configure priorities
- Configure priority with hardware
- Interrupt the shield

8.4.2 Interruption requests are open and prohibited

- Many times, we need to be able to prevent interruptions in running programs, so we must provide mechanisms to prevent interruptions from occurring, or ignore them for the time being.
- Levels of prohibition of interruptions：
  - Interrupt requests from a special device are prohibited.
  - Outage requests from lower or the same priority are not accepted.
  - Interrupt requests from all interrupt sources are prohibited.

8.4.2 Interruption requests are open and prohibited

- Interrupt requests from a special device are prohibited：
  - This degree of shutdown is relatively simple, as long as the interrupt inhibit bit of the device is set to 1, the device cannot interrupt the execution of any program.

8.4.2 Interruption requests are open and prohibited

- Outage requests from lower or the same priority are not accepted：
  - Computer systems can have multiple sources of interruptions, each given different interrupt priorities due to their importance. In this case, when the CPU is processing a higher priority interrupt, it must prohibit interrupts of lower or equal importance to interrupt. On some computer systems, an interrupt mask register is used to represent that an interrupt cannot be interrupted by other interrupts, so as long as the bits of the mask are set correctly, the purpose can be achieved.

8.4.2 Interruption requests are open and prohibited

- Interrupt requests from all interrupt sources are prohibited：
  - This level of interruption prohibition can be achieved in two ways. The first is provided by the hardware itself, which can temporarily block all interrupt interference, so that the CPU can execute at least one more command after acknowledging one interrupt before opening other interrupts; This mechanism ensures that the CPU can always execute the first instruction of the previous interrupt service routine. If it is necessary to execute several more commands for the interrupt service routine, such as temporarily saving the execution environment of a previously running program, then its first instruction must be a "Prevent all interrupts" command; Using instructions like this to prohibit all interrupts is the second method, which forces the CPU to ignore all interrupt requests. However, whether you use hardware or software to prohibit interrupt requests, in fact, it only allows the CPU to temporarily not respond to interrupt requests.

8.4 Priority of interruptions

- Nested break
- Interruption requests are open and prohibited
- Use software to configure priorities
- Configure priority with hardware
- Interrupt the shield

8.4.3 Use software to configure priorities

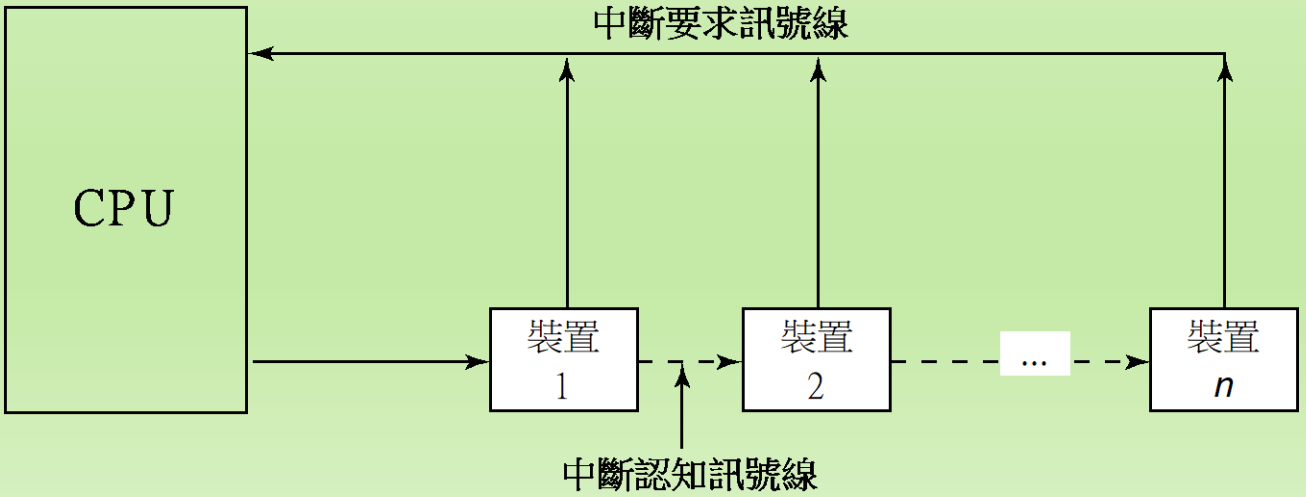
- A method that uses software (polling routines) to identify the source of the outage：
  - The order of polling is equivalent to the priority of the interrupt.
  - When either device makes an interrupt request, the polling software starts running.
  - An accepted source of interrupt is not necessarily the device that first made the interrupt request.
  - Easy to change the priority of interruptions序：
    - Advantage：Great elasticity.
    - Disadvantage：Poor efficiency.

8.4 Priority of interruptions

- Nested break
- Interruption requests are open and prohibited
- Use software to configure priorities
- Configure priority with hardware
- Interrupt the shield

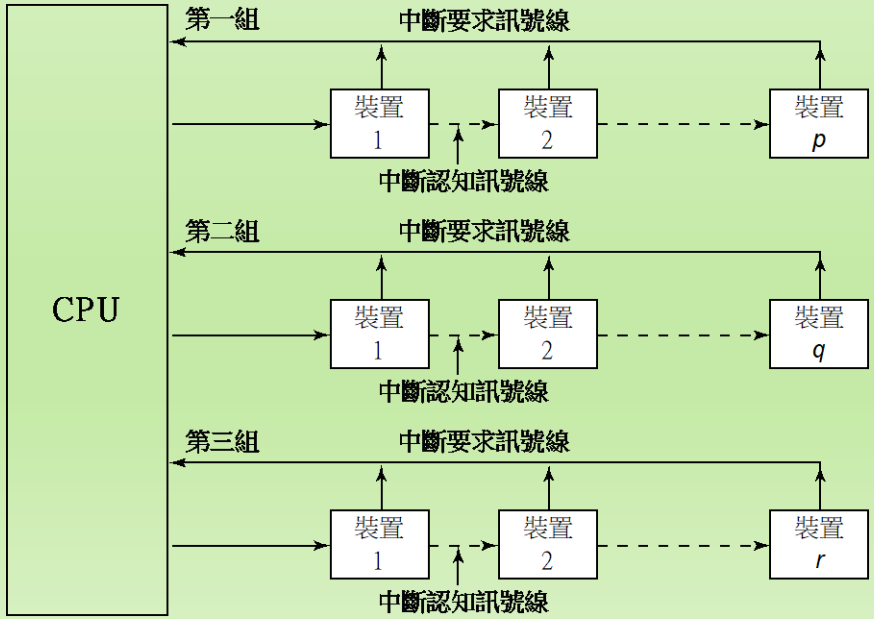
8.4.4 Configure priority with hardware

- Daisy chain：
  - The recognition signal line is connected to the device with the highest priority first, and then from the first device to the device with the next highest priority.
  - When an interrupt acknowledgment message is sent to any device that has the mid-request flag turned on, the message is intercepted.



8.4.4 Configure priority with hardware

- multi-level：
  - When a computer provides multiple interrupt request lines, a set of priority assignment circuits determines which device on the interrupt request line the CPU should serve.
  - Use the cascading design to determine which device to accept interrupt requests on individual lines.



8.4 Priority of interruptions

- Nested break
- Interruption requests are open and prohibited
- Use software to configure priorities
- Configure priority with hardware
- Interrupt the shield

8.4.5 Interrupt the shield

- interrupt masking：
  - Multiple interrupt lines are used to connect to the CPU, and a special interrupt register is designed in the CPU, so that each interrupt line corresponds to a different bit on the interrupt register
    - When any device makes an interrupt request, the corresponding bit is automatically set to 1 。
  - Use the interrupt mask register to change the priority of the interrupt
    - The program sets the bit corresponding to the interrupt request to be prohibited to 0.

8.4.5 Interrupt the shield

- The CPU will perform AND operations on this mask and interrupt register：
  - The corresponding bit of the prohibited device becomes 0.
  - Acceptable interruptions are allowed to stay
- Advantages of using masked interrupts：
  - Change the priority of interruptions at any time like software.
  - Better performance like hardware.
  - Flexibility and performance.

The End